Analiza probabilistyczna algorytmów

Przykład:
zatrudnianie pracownika ("problem sekretarki").

Dokonać analizy poniższego algorytmu realizującego procedurę zatrudniania nowego pracownika.

Algorytm:

- rozmowa kwalifikacyjna z kolejnymi kandydatami,
- jeśli kolejny kandydat lepszy niż wszyscy poprzedni to go zatrudniaj, zwalniając ostatnio zatrudnionego,
- rozmawiaj z następnym, i tak do końca listy kandydatów.

```
najlepszy := -∞ ;          // dotychczas zatrudniony
for i:=1 to n do
   przesłuchaj kandydat[i];
   if kandydat[i] lepszy niż najlepszy then
      zwolnij najlepszy;
      najlepszy := i;
      zatrudnij najlepszy;
end.
```

Koszt algorytmu $\stackrel{\text{def}}{=}$ liczba operacji "zwolnij/zatrudnij".

"Normalne" sformułowanie problemu:
- szukamy największego elementu standardowym algorytmem przeglądającym kolejne elementy listy;
- pytanie: ile razy będziemy zapamiętywać kolejny element jako dotychczasowy największy.

Przypadek pesymistyczny:

- kandydaci w kolejności rosnącej;
- liczba operacji zwolnij/zatrudnij = n.

Analiza probabilistyczna

Może dotyczyć kosztu całego algorytmu albo wybranych instrukcji.

- dokonaj założenia o rozkładzie prawdopodobieństwa;
- oblicz wartość oczekiwaną czasu wykonania.

Dla problemu zatrudniania zakładamy:

- parami różne i porównywalne kwalifikacje wszystkich kandydatów,
- najprościej: każdy kandydat ma kwalifikację z zakresu [1..n],
- kolejność kandydatów losowa, tzn.
  - na wejściu permutacja losowa liczb 1..n,
  - czyli jedna z możliwych n! permutacji,
  - każda z prawdopodobieństwem 1/n!.

Randomizacja

Wprowadzenie losowości do danych wejściowych;
W problemie zatrudniania może to być np.:

- przed każdym przesłuchaniem losuj nowego kandydata.

Technicznie: zakładamy dostępność funkcji RAND

k := RAND(i, j), i, j, k - całkowite

zwraca liczbę całkowitą z przedziału [i,j], każdą z jednakowym prawdopodobieństwem.

W szczególności:

RAND(0,1) $\Leftrightarrow$ rzut symetryczną monetą.

Zmienne losowe wskaźnikowe

- zbiór zdarzeń elementarnych S;
- zdarzenie $A \subseteq S$;

Zmienna losowa wskaźnikowa (indykator) zdarzenia A:

$I\{A\} \stackrel{def}{=}$ 1, jeśli zachodzi A,
0, wpp.

Lemat
S – przestrzeń zdarzeń elementarnych, A – zdarzenie, $X_A = I\{A\}$. Wtedy $E[X_A] = Pr\{A\}$.

Dowód oczywisty:

$E[X_A] = E[I\{A\}]$
$= 1*Pr\{A\} + 0*Pr\{\overline{A}\}$ // $\overline{A} \stackrel{def}{=} S - A$
$= Pr\{A\}$ QED

Przykład zastosowania:
oblicz oczekiwaną liczbę orłów w n rzutach monetą.

$X \stackrel{def}{=}$ "liczba orłów w n rzutach"; // zmienna losowa

Obliczanie wprost z definicji zmiennej losowej:

$E[X] = \sum_{i=0..n} i * Pr\{X=i\}$

jest nieco skomplikowane. Indykatorami prościej.

$X_i \stackrel{def}{=} I\{$w i-tym rzucie wypadł orzeł$\}$
// zmienna losowa wskaźnikowa

Obserwacja 1: $E[X_i] = 1/2, \quad i=1,...,n.$

Obserwacja 2: X_i = "liczba orłów w i-tym rzucie".

Wniosek: $X = \sum_{i=1..n} X_i$.

Stąd:

$E[X] = E[\sum_{i=1..n} X_i]$

// addytywność wartości oczekiwanej

$= \sum_{i=1..n} E[X_i]$

$= \sum_{i=1..n} 1/2 = n/2$.

Analiza algorytmu zatrudniania

za pomocą zmiennych losowych wskaźnikowych

$X \stackrel{def}{=}$ zmienna losowa "liczba kandydatów zatrudnianych i zwalnianych".

Z definicji: $E[X] = \sum_{i=1..n} i * Pr\{X=i\}$,

ale obliczenie $Pr\{X=i\}$ jest kłopotliwe.

$X_i \stackrel{def}{=} I\{$kandydat i-ty jest zatrudniany$\}$

Lemat $\Rightarrow$ $E[X_i] = Pr\{$kandydat i-ty jest zatrudniany$\}$.

Fakt: Kandydat i-ty jest zatrudniany wtedy i tylko wtedy, gdy jest najlepszy spośród kandydatów 1..i.

Permutacja n kandydatów jest losowa, zatem

początkowe i elementów na wejściu jest w kolejności losowej,

stąd: $Pr\{$kandydat i-ty jest najlepszy$\} = 1/i$.

Fakt: $X = \sum_{i=1..n} X_i$.

$E[X] = E[\sum_{i=1..n} X_i] = \sum_{i=1..n} E[X_i]$

// addytywność wartości oczekiwanej

$= \sum_{i=1..n} 1/i = \ln n + O(1)$.

skorzystaliśmy z własności szeregu harmonicznego:
n-ta liczba harmoniczna

$H_n \stackrel{def}{=} \sum_{i=1..n} 1/i = \ln n + \gamma + O(1/n), \quad \gamma \approx 0.57$ *– stała Eulera*

Wniosek:
Oczekiwany (średni) koszt algorytmu zatrudniania (liczba operacji zwolnij/zatrudnij) wynosi $\Theta(\log n)$.

Algorytmy randomizowane

Gdy

- nie mamy wiedzy o rozkładzie prawdopodobieństwa wśród danych wejściowych, albo
- rozkład jest niewygodny (chcemy aby był lepszy - obliczenia były szybsze),

możemy narzucić rozkład prawdopodobieństwa za pomocą randomizacji.

Przykład: dla problemu zatrudniania konstruujemy algorytm "randomizowane zatrudnianie":

Dodajemy randomizację danych wejściowych na początku dotychczasowej wersji.

```
dokonaj losowej permutacji tablicy kandydat[1..n];
najlepszy = -∞ ;        // dotychczas zatrudniony
... dalej identycznie jak poprzednio ...
end.
```

W ten sposób otrzymujemy algorytm, którego analiza (zatem również oczekiwana złożoność) nie zależy od rozkładu danych wejściowych.

Uwaga:

- dla ustalonych danych wejściowych, algorytm deterministyczny wykonuje zawsze tę samą sekwencję operacji,
- algorytm randomizowany wykonuje różne ciągi operacji dla różnych wykonań tego algorytmu (zależne od wyniku losowania) na tych samych danych wejściowych,

zatem:

- dla algorytmu randomizowanego nie istnieją dane pesymistyczne (w sensie: nie da się przygotować danych pesymistycznych),
- każde dane wejściowe mogą okazać się pesymistyczne.

Generowanie permutacji losowej
czyli losowanie permutacji,

każda z n! możliwych permutacji powinna być wynikiem losowania z prawdopodobieństwem 1/n!

Algorytm "Knuth's shuffle" *// tasować*
(Knuth, Fisher-Yates)

```
tablica A[1..n];
for i:=1 to n do
   swap(A[i], A[RAND(i,n)]);
end.
```

Lemat.
Algorytm generuje losową permutację tablicy A.

Definicja:

k-permutacją ze zbioru n-elementowego jest dowolny k-elementowy ciąg parami różnych elementów tego zbioru

inne określenie: wariacje bez powtórzeń

Fakt:

Liczba różnych k-permutacji w zbiorze n-elementowym wynosi n!/(n-k)!

Dowód lematu

Niezmiennik: przed rozpoczęciem iteracji i-tej, dla każdej możliwej (i-1)-permutacji σ,

$\Pr\{A[1..i-1] = \sigma\} = (n-i+1)!/n!$.

czyli każda (i-1)-permutacja na początku tablicy jest jednakowo prawdopodobna.

Dowód poprawności niezmiennika:

inicjalizacja:

i=1, A[1..0] jest pusta, zawiera więc pustą permutację z prawdopodobieństwem 1,
z drugiej strony: (n-1+1)!/n! = 1 $\Rightarrow$ OK.

krok indukcyjny $i \rightarrow i+1$:

zawartość A po i-tej iteracji: $A[1..i] = [x_1,...,x_i]$,
gdzie:

$x_1,...,x_{i-1}$ – permutacja po iteracji (i-1)-szej,
x_i – element dodany w iteracji i-tej w A[i].

$E_1 \stackrel{\text{def}}{=}$ zdarzenie polegające na tym, że początkowe i-1 iteracji wygenerowało $x_1,...,x_{i-1}$.

$\Pr\{E_1\} = (n-i+1)!/n!$, z niezmiennika (zał. ind.)

$E_2 \stackrel{def}{=}$ zdarzenie polegające na tym, że i-ta iteracja wstawiła x_i do A[i].

Należy obliczyć $Pr\{E_1 \cap E_2\}$.

Z prawdopodobieństwa warunkowego:

$$Pr\{E_2 \cap E_1\} = Pr\{E_2|E_1\} * Pr\{E_1\}.$$

$$Pr\{E_2|E_1\} = 1/(n-i+1),$$

bo x_i wybierane losowo spośród A[i..n]. Zatem:

$$Pr\{E_2 \cap E_1\} = [1/(n-i+1)] * [(n-i+1)!/n!]$$
$$= (n-i)!/n!$$

krok indukcyjny OK

Zakończenie:

dla i=n+1, czyli po iteracji i-tej mamy:

A[1..n] zawiera n-elementową permutację wylosowaną z prawdopodobieństwem

$$(n-(n+1)-1)!/n! = 1/n!$$

QED

Inne metody generowania permutacji losowych:

```
1.
// losowa permutacja liczb 1..n
for i:=1 to n do
   repeat
      k := RAND(1,n);
   until k nie występuje w A[1..i-1];
   A[i] := k;
```

losowość? oczekiwany czas?

2.

```
// losowa permutacja tablicy A[1..n]
for i:=1 to n do
   P[i] := RAND(1,n^3)
posortuj A, kluczami są wartości P[1..n];
   // problem: usunąć powtórzenia kluczy
```

losowość? prawdopodobieństwo powtórzeń?

Przykład analizy probabilistycznej: problem zatrudniania on-line

Założenia:

- chcemy zatrudnić tylko raz,
- po każdej rozmowie natychmiast poinformować kandydata czy zostaje zatrudniony,
- godzimy się z tym, że nie zawsze uda się nam zatrudnić najlepszego ze wszystkich.

Jaką strategię wyłaniania kandydata przyjąć?

Jakie będzie prawdopodobieństwo zatrudnienia najlepszego przy tej strategii?

Strategia:

- pierwszych k kandydatów nie zatrudniaj;
- spośród kandydatów następnych zatrudnij pierwszego, który jest większy niż największy spośród początkowych k kandydatów;
- jeśli największy kandydat jest wśród pierwszych k kandydatów to zatrudnij ostatniego.

```
best = -∞ ;
for i:=1 to k do
   if A[i] > best then
      best := A[i];
for i:=k+1 to n do
   if A[i] > best then
      return i;
return n;
```

Zauważmy:

- pierwszą część, przegląd początkowych k kandydatów, można nazwać "fazą uczenia się",
- prawdopodobieństwo wyboru kiepskiego kandydata (a nawet najgorszego) jest niezerowe.

Problem:

Znajdź k takie, że prawdopodobieństwo wyboru najlepszego kandydata jest największe i oszacuj (od dołu) to prawdopodobieństwo.

Niech k ustalone.

$$\Pr\{\text{wybraliśmy najlepszego}\} = ?$$

Niech $M(j) \stackrel{\text{def}}{=} \max\{A[1..j]\}$, $M(n) = \max A$.

- $S \stackrel{\text{def}}{=}$ zdarzenie "wybraliśmy najlepszego",
- $S_i \stackrel{\text{def}}{=}$ zdarzenie "wybraliśmy najlepszego, jest to A[i]";
- oczywiście $S = S_1 \cup \ldots \cup S_n$.

Fakt.

$S_i \cap S_j = \varnothing$, $i \neq j$, oraz $\Pr\{S_i\} = 0$, $i=1,\ldots,k$,

bo jeśli max jest w 1..k, to go nie wybierzemy, więc

$$\Pr\{S\} = \Sigma_{i=k+1..n} \Pr\{S_i\} .$$

Aby zaszło S_i muszą zachodzić następujące zdarzenia:

1. $B_i \stackrel{def}{=}$ "A[i] = max A", oraz
2. O_i := "nie wybraliśmy żadnego z A[k+1..i-1], czyli A[j] < M(k), j=k+1,...,i-1" .

```
|-  -  -  -  -  -| -  -  -  -  - max -  -  -  -  -|
 1               k k+1          i-1 i              n
```

Fakt: B_i , O_i - niezależne. Dowód kombinatoryczny:

- kolejność na pozycjach 1..i-1 nie ma wpływu na to że wartość na pozycji i-tej jest od nich większa,
- wartość na pozycji i-tej nie wpływa na kolejność na pozycjach 1..i-1.

Inaczej: to że A[i]=max nie ma żadnego związku z kolejnością w A[1..i-1].

Zatem $Pr\{S_i\} = Pr\{B_i \cap O_i\} = Pr\{B_i\} * Pr\{O_i\}$

$Pr\{B_i\} = 1/n$,

$Pr\{O_i\} = k/(i-1)$,

bo to zdarzenie oznacza że najlepszy spośród kandydatów 1..i-1 występuje wśród 1..k .

$$\begin{aligned} Pr\{S\} &= \Sigma_{i=k+1..n}\, Pr\{S_i\} = \Sigma_{i=k+1..n}\, Pr\{B_i\} * Pr\{O_i\} \\ &= \Sigma_{i=k+1..n}\, k/(n(i-1)) \\ &= k/n\ \Sigma_{i=k+1..n}\, 1/(i-1) \\ &= k/n\ \Sigma_{i=k..n-1}\, 1/i \end{aligned}$$

Funkcja 1/x jest malejąca, więc

$$\int_{[k,n]} 1/x\, dx \ \le\ \Sigma_{i=k..n-1}\, 1/i \ \le\ \int_{[k-1,n-1]} 1/x\, dx$$

$\int 1/x\, dx = \ln x + C$, zatem

$$(k/n)(\ln n - \ln k) \leq \Pr\{S\} \leq (k/n)(\ln(n-1) - \ln(k-1))$$

Przy ustalonym n są to funkcje zmiennej k.

Maksymalizujemy prawdopodobieństwo, więc ograniczamy od dołu, szukając maksimum dla dolnego ograniczenia. Pochodna po zmiennej k lewej części wynosi:

$$(1/n)(\ln n - \ln k) + (k/n)(-1/k)$$
$$= (1/n)(\ln n - \ln k - 1)$$

przyrównując do 0:

$$\ln k = \ln n - 1 \Rightarrow \ln k = \ln (n/e),$$

zatem pochodna wynosi 0 dla $k = n/e$.

Pochodna w $k=n/e$ zmienia znak z dodatniego na ujemny, więc rzeczywiście mamy maksimum, równe $1/e$.

Wniosek:
Dla $k \approx n/e$ prawdopodobieństwo zatrudnienia najlepszego kandydata jest największe i wynosi co najmniej $1/e$.

Przykład analizy za pomocą zmiennych losowych wskaźnikowych - quicksort (Lomuto)

Dane: permutacja losowa liczb 1,...,n w tablicy A[1..n]

```
Partition(A, L, R)   // wersja Nico Lomuto
   v := A[R];        // ostatni element jest dzielący
   i := L-1;
   for j:=L to R-1 do
           // Inv: A[L..R]=[L...i,i+1...j-1,j...R-1,R]
           //               {<=v} {   >v   }{  ??? } v
      if A[j]<=v then
         i := i+1;
         swap (A[i], A[j]);
   swap (A[i+1], A[R]);
   return i+1;
end;

quicksort(A, L, R)
   if (L < R) then
      p := Partition(A, L, R);
      Quicksort(A, L, p-1);
      Quicksort(A, p+1, R);
end;

main: quicksort(A, 1, n);
```

Fakt.
Dla dowolnych elementów i<j, są one porównywane ze sobą w całym algorytmie co najwyżej raz.

Bo:

- jeśli są porównywane, to jeden z nich jest pivotem;
- pivot danego podzadania w dalszej części algorytmu już nie jest porównywany.

Uwaga: dla wersji Hoare'a Fakt nie zachodzi.

$X_{i,j} \stackrel{def}{=}$ I{i porównywane z j} // kolejność i,j obojętna

$X_n \stackrel{def}{=}$ liczba porównań

$X_n = \sum_{i=1,..,n-1} \sum_{j=i+1,..,n} X_{i,j}$.

$E[X_n] = E[\sum_{i=1,..,n-1} \sum_{j=i+1,..,n} X_{i,j}]$

$= \sum_{i=1,..,n-1} \sum_{j=i+1,..,n}$ Pr{i porównywane z j}

Fakt 4.
Dla dowolnych i<j, i jest porównywane z j wtedy i tylko wtedy, gdy w przedziale wartości [i,j] pierwszym elementem, który w algorytmie stał się pivotem, jest i lub j.

Dowód.

- na początku i oraz j są w jednym podzadaniu;
- trafiają do różnych podzadań, gdy któryś z elementów k, i<k<j, staje się pivotem;
- jeśli to się stanie, i oraz j nigdy nie będą ze sobą porównywane.

QED

Fakt 5.
Pr{pierwszym pivotem z [i..j] został i} = 1/(j-i+1).

Dowód.

- żaden element k, i≤k≤j, nie był jeszcze pivotem, więc przedział [i,j] zawiera się w aktualnym podzadaniu;
- w tym podzadaniu, przy warunku, że wybieramy z przedziału [i,j], prawdopodobieństwo wyboru i wynosi 1/(j-i+1).

QED

Wybór i jako pierwszego pivota jest zdarzeniem rozłącznym z wyborem j jako pierwszego.

Zatem:

$$\Pr\{i \text{ porównywane z } j\} = 2/(j-i+1).$$

$$E[X_n] = \sum_{i=1,..,n-1} \sum_{j=i+1,..,n} 2/(j-i+1) \quad // \quad j-i = k$$
$$= 2 \sum_{i=1,..,n-1} \sum_{k=1,..,n-i} 1/(k+1)$$

w każdej $\sum_{k\ldots}$ dodajemy 1 i resztę szeregu do 1/n:

$$E[X_n] < 2 \sum_{i=1,..,n-1} \sum_{k=1,..,n} 1/k$$
$$= 2(n-1)H_n = 2n \ln n + O(n).$$

Jeśli chcemy dokładniej, odejmujemy dodane jedynki:

$$E[X_n] < 2n \ln n - 2(n-1) + O(n).$$

W składniku O(n) współczynnik pochodzi ze stałej Eulera, pomnożonej przez 2, zatem jest mniejszy niż 2. Otrzymujemy:

$$E[X_n] = 2n \ln n - f(n),$$

gdzie $0 < f(n) \leq cn$, $c<1$.